Claude Code 구현 가이드

시승·견적 신청 시스템 구현

재규어·랜드로버 6개 차종으로 만드는 고객 신청부터 견적 발행까지

프롬프트 중심 · 누구 → 어디 → 입력 → 출력 → 피드백

사람은 프롬프트를 붙여넣고 화면을 확인한다. 설계·코딩·오류 수정·테스트는 Claude Code가 한다.
모든 가격·전시장·고객 데이터는 교육용 가상 자료다.

이번 구현의 핵심은 세 가지다

무엇을, 누가, 어디까지 만드는가

1

고객은 시승·견적·둘 다 중 하나를 골라 신청한다

차량 선택 → 유형 선택 → 일정 또는 견적 조건 → 연락처·동의 → 이메일 인증 후 접수

2

사람은 프롬프트와 확인, Claude Code는 나머지 전부

설치·코딩·DB·테스트·배포 준비를 Claude가 한다. 로그인·비밀키 입력·화면 확인만 사람이 한다

3

완성 기준은 '완료 화면'이 아니라 '저장·권한·중복 차단'이다

DB에 저장되고, 담당자만 처리하고, 고객은 본인 것만 보고, 같은 시간 중복 예약이 막혀야 한다

접수는 확정이 아니다 — 담당자 확인 후 시승이 확정되고 견적이 발행된다

완성 모습

고객 신청 7단계가 담당자 처리로 이어진다

고객 ①~⑤ 신청 → 담당자 ⑥ 처리 → 고객 ⑦ 확인. 참고 화면: jaguar-land-rover-experience.ai.studio



접수는 확정이 아니다 — 담당자 확인 후 시승 확정·견적 발행

실습은 첨부한 6개 차종과 사진으로 진행한다

사진은 Kit의 reference-assets, 차종 매핑은 data/assets-map.json



Range Rover



Range Rover Sport



Range Rover Velar



Discovery



Defender 110



Jaguar I-PACE

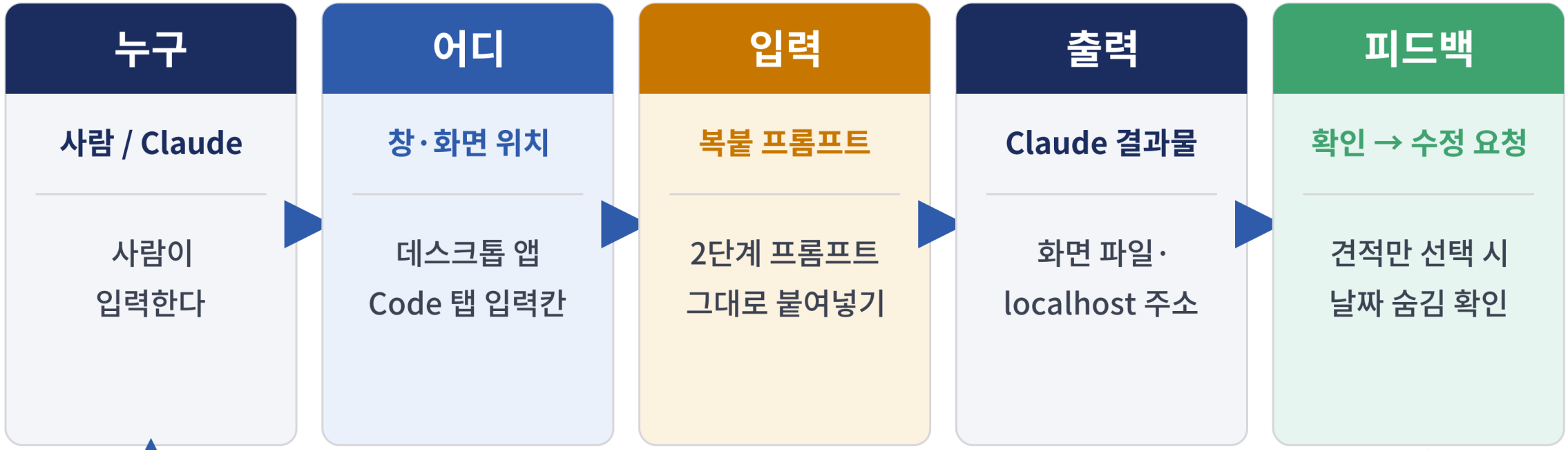
신청 수집부터 견적까지 한 시스템에서 처리된다

자동차 판매 업무의 AX — AI는 개발과 견적 초안을 돕고, 가격·할인·확정은 담당자가 검토

기존 업무	구현 후	확인 지표
전화·메신저로 신청 수집	한 가지 양식으로 접수	접수 누락 건수
달력과 차량 현황을 따로 확인	가능한 시승 슬롯만 표시	중복 예약 건수
상담 조건을 다시 질문	차량 구성·구매 조건 함께 저장	재확인 횟수
견적서를 매번 새로 작성	조건으로 초안 → 담당자 확정	견적 작성 시간
진행 상황을 개인별 관리	접수·연락·시승·견적 상태 공유	최초 응답 시간

모든 단계는 누구·어디·입력·출력·피드백으로 움직인다

앰버 박스 = 붙여넣을 프롬프트, 네이비 박스 = Claude가 만든 결과, 그린 패널 = 확인과 수정 요청



피드백이 다음 입력이 된다 — 모든 단계가 이 다섯 칸으로 움직인다

사람의 몫은 계정·비밀키·확인 세 가지뿐이다

나머지는 모두 Claude Code에게 시킨다

사람이 직접 한다

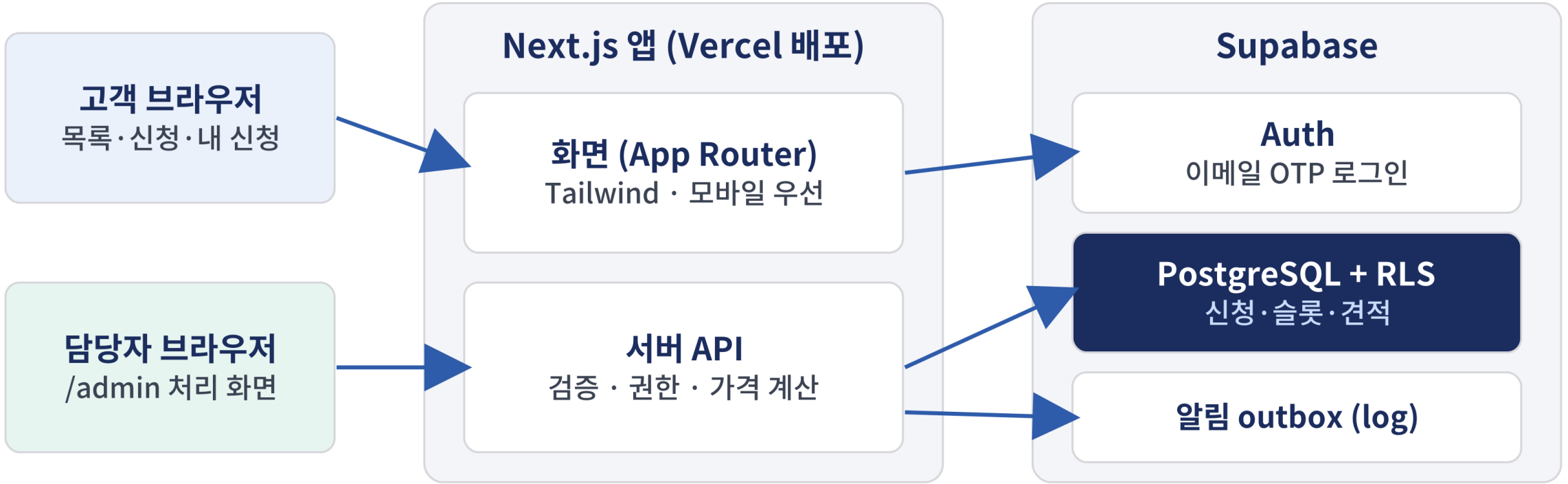
- Claude 데스크톱 앱 ·Supabase·GitHub·Vercel 로그인
- 비밀키를 .env.local에 직접 붙여넣기
- 권한 요청을 읽고 허용
- 브라우저로 화면 확인 → 피드백

Claude Code가 한다

- 개발환경 점검·패키지 설치
- 요구사항 문서·화면·DB·API 작성
- 오류 재현과 수정
- 테스트·빌드·배포 준비와 보고

Next.js와 Supabase 두 축으로 시스템을 구성한다

Next.js + TypeScript + Tailwind CSS + Supabase — 버전은 Claude가 공식 문서로 확인해 고른다



비밀키는 서버에만 — 브라우저는 publishable key와 RLS로만 데이터에 닿는다

12개 프롬프트를 3구간으로 나눠 진행한다

한 번에 한 단계 — 결과를 확인한 뒤 다음 프롬프트를 넣는다

화면 실습

- 0 공통 지시
- 1 요구사항
- 2 고객 화면

작동하는 MVP

- 3 Supabase 연결
- 4 DB·권한
- 5 이메일 로그인
- 6 접수·중복 방지

운영 준비

- 7 관리자
- 8 견적·PDF
- 9 알림
- 10 테스트
- 11 배포

가상 데이터와 업무 문서를 키트로 제공한다

JLR_TestDrive_Kit 폴더를 프로젝트 폴더에 통째로 복사한다 — 파일은 Claude가 읽는다

파일	내용
reference-assets/	차량 사진 12장 · contact_sheet · 원본 출처
data/vehicles · trims · options.csv	6개 차종, 트림 3종, 색상·패키지 (가상 가격)
data/retailers · test_drive_resources.csv	가상 전시장 4곳, 시승 차량 13대
data/slot_rules · staff · sample_applications.csv	슬롯 규칙, 가상 직원 7명, 샘플 신청 20건
data/assets-map.json	사진 → 차종 매핑과 확신도
docs/business_rules · quote_policy.md	상태 전이·시승 규칙, 견적 계산·할인 한도
docs/consent_privacy_v1 · quote_sample.md	가상 동의서 문안, 검산용 견적서 샘플

01

STEP 0 · 준비

작업 폴더와 공통 원칙

앱 설치 · 폴더 연결 · 환경 점검 · CLAUDE.md

앱 설치와 폴더 연결은 사람이 한 번만 한다

계정은 Claude Code를 쓸 수 있는 요금제로 준비한다

누구 사람이 직접 한다

어디 브라우저 · 파일 탐색기 · Claude 데스크톱 앱

① 앱 설치

claude.com/download

Claude 데스크톱 앱을
설치하고 로그인한다

Code 탭 확인

② 폴더 만들기

문서/AX/jlr-experience

빈 프로젝트 폴더를 만든다

영문 폴더명 권장

③ 키트 복사

JLR_TestDrive_Kit

압축을 풀어 폴더째
프로젝트 폴더 안에 넣는다

사진·CSV·문서 포함

④ Code 탭 연결

폴더 선택

Code 탭에서 프로젝트
폴더를 선택한다

이후 모든 입력은 여기에

Supabase·GitHub·Vercel 계정은 해당 단계에서 만들어도 된다

프롬프트는 Code 탭 입력칸에 붙여넣는다

이 덱의 '어디' 칸이 가리키는 곳 — 폴더를 먼저 고르고, 권한 요청은 읽고 허용한다



메뉴 이름과 배치는 앱 버전에 따라 조금씩 다를 수 있다

STEP 0-1

개발환경 점검과 설치 안내도 Claude에게 맡긴다

Node.js·Git이 없으면 Claude가 운영체제에 맞는 설치 순서를 알려 준다

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
나는 코딩 초보자다. 이 폴더에서 자동차
시승·견적 신청 웹앱을 만들 것이다.
먼저 개발환경만 점검해줘.

1. node, npm, git 버전을 확인해줘.
2. 없거나 오래됐으면 내 운영체제에 맞는
   설치 방법을 클릭 순서로 알려줘.
3. JLR_TestDrive_Kit의 README.md와
   파일 목록을 읽고 무엇이 있는지 요약해줘.
4. 아직 앱은 만들지 마.
```

[출력] Claude가 만드는 것

```
node v22.x · npm 10.x · git 2.x
설치 안내 (필요한 것만)
키트 구성 요약 (29개 파일)
```

피드백 · 확인 → 수정 요청

- 세 버전이 모두 숫자로 나오는가
- 설치했다면 '다시 확인해줘'
- 권한 요청은 내용을 읽고 허용

STEP 0-2

공통 원칙을 CLAUDE.md에 저장해 매 단계에 적용한다

CLAUDE.md = Claude Code가 이 폴더에서 매번 먼저 읽는 작업 규칙 파일

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
이 폴더에 시승·견적 신청 시스템을 구현한다.
참고: jaguar-land-rover-experience.ai.studio
자료: JLR_TestDrive_Kit (사진·데이터·규칙)
아래 원칙을 CLAUDE.md에 저장하고 계속 지켜줘.

1. 설명은 한국어, 내가 할 일은 클릭 순서로.
2. 설치·코딩·실행·검증은 네가 직접 한다.
3. 요청한 단계만 끝내고 결과·주소·확인 항목을
   보고한 뒤 다음 지시를 기다린다.
4. Next.js App Router·TypeScript·Tailwind·
   Supabase. 버전은 공식 문서로 확인한다.
5. 비밀키는 .env.local에만. 출력·요청 금지.
6. 권한은 서버가 검증. DB 변경은 migration.
7. 가상 데이터만. 서버 저장 전 '접수 완료' 금지.
8. 단계마다 lint·타입·테스트·빌드, 미실행 명시.
```

[출력] Claude가 만드는 것

```
CLAUDE.md
환경 점검 결과
다음 단계 대기 메시지
```

피드백 · 확인 → 수정 요청

- CLAUDE.md를 열어 8개 원칙 확인
- 빠진 원칙은 '추가해줘'
- 작업 폴더 경로가 맞는가

02

STEP 1–2 · 화면

요구사항 문서와 고객 화면

DB 없이 가상 데이터로 화면부터 완성한다

STEP 1

구현 전에 요구사항을 문서로 확정한다

키트의 업무 규칙·견적 정책·동의서를 근거로 쓴다

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
1단계: docs/requirements.md와
docs/acceptance-checklist.md를 작성해줘.
근거: Kit의 docs/business_rules.md,
quote_policy.md, consent_privacy_v1.md

고객: 차량 목록·상세 → 유형(test_drive /
quote / both) → 시승 일정 또는 견적 조건
→ 연락처·동의 → 이메일 인증 후 접수 → 내 신청
관리자: 목록·필터·담당자·메모, 시승 확정/
완료/노쇼/취소, 견적 초안·발행·버전·PDF

시승 상태와 견적 상태는 따로 설계한다.
접수는 확정이 아니다. 교육용임을 표시한다.
이번에는 문서까지만.
```

[출력] Claude가 만드는 것

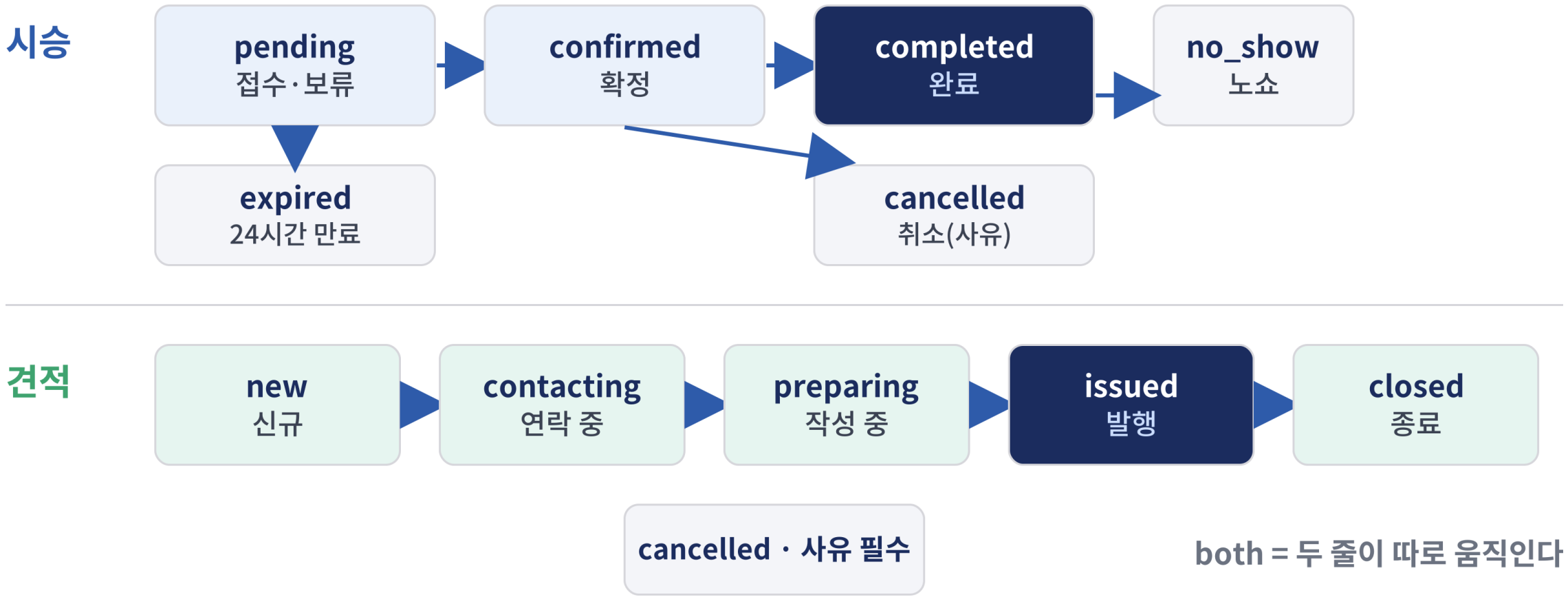
```
docs/requirements.md
docs/acceptance-checklist.md
기술 선택 이유·단계별 완료 기준
```

피드백 · 확인 → 수정 요청

- quote에 날짜 필수가 없는가
- both의 상태가 두 줄인가
- '접수=확정' 문구가 없는가

시승 상태와 견적 상태는 독립적으로 움직인다

문서의 상태도가 이 그림과 같은지 확인한다 — 다르면 '이 그림대로 고쳐줘'



STEP 2

고객 화면은 DB 없이 가상 데이터로 먼저 완성한다

adapter = 데이터를 읽는 층. 나중에 DB로 바꿔 끼운다

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
2단계: 고객 화면을 만들고 로컬에서 실행해줘.
DB 없이 Kit의 data/*.csv를 읽는 가상 adapter.
화면: / 목록, /vehicles/[slug] 상세, /apply,
/apply/success, /my, /admin(데모 골격)
사진: data/assets-map.json대로 public/images에
복사. confidence=low는 '이미지 확인 필요'.
디자인: 딥그린 #002a15, 아이보리 #fcf9f8,
Pretendard(없으면 시스템 폰트), 입력 16px+.
상세의 '이 차량으로 신청'이 차량을 넘긴다.
유형별 필드·필수 동의·오류 메시지·진행 표시.
완료 화면: '데모: 실제 접수되지 않았습니다'.
개발 서버를 켜고 주소와 확인 순서를 알려줘.
```

[출력] Claude가 만드는 것

```
app/ 화면 6개 · public/images 12장
http://localhost:3000
확인 순서 안내
```

피드백 · 확인 → 수정 요청

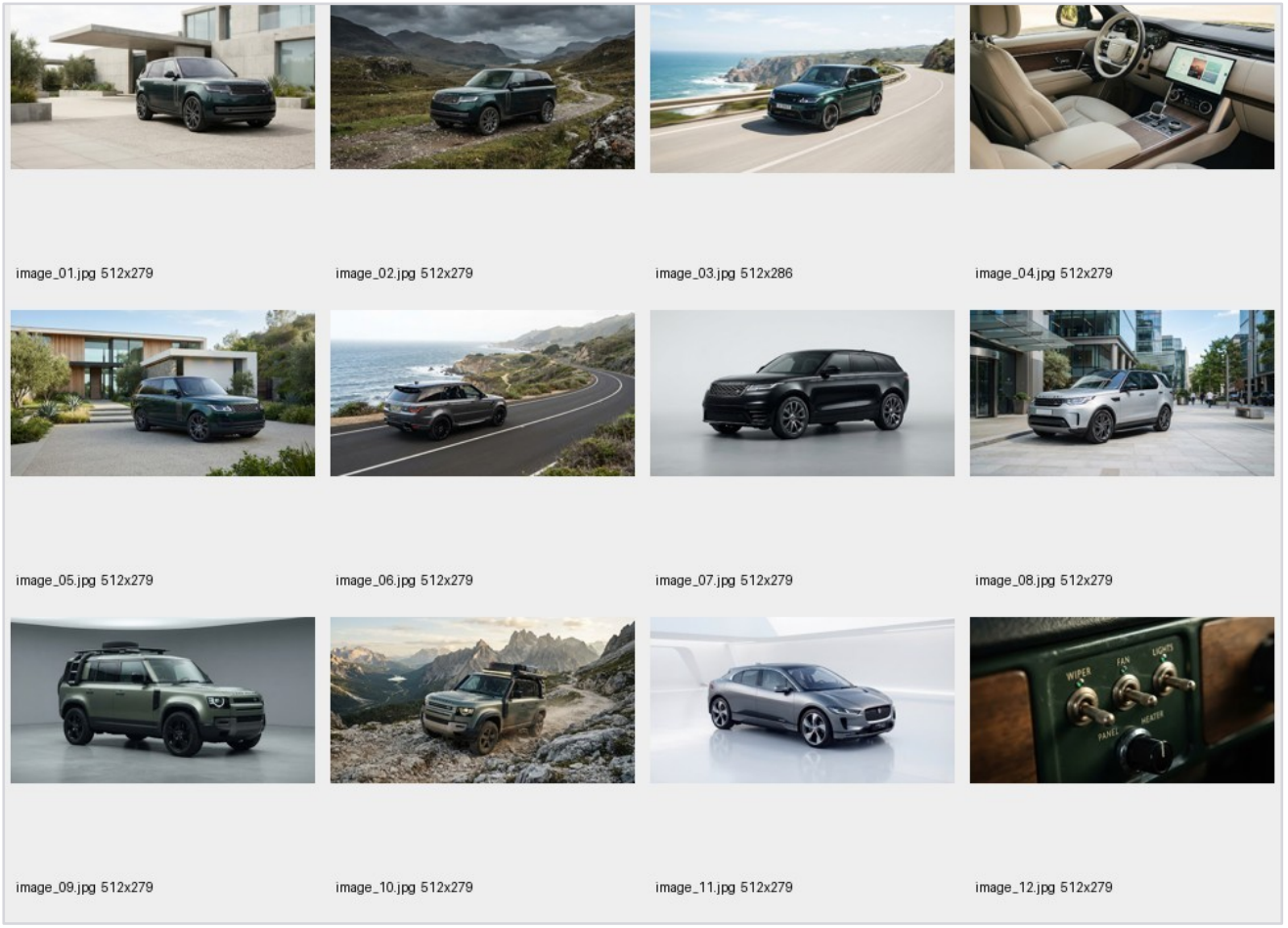
- 주소를 브라우저로 연다
- 차종 6개가 사진과 함께 보이는가
- 확인 순서는 다음 장

사진과 차종 매핑을 눈으로 확인한다

Claude가 사진을 보고 만든 매핑이 맞는지 사람이 최종 판단한다

확인할 것

- 주 사진 6장
- 실내·스위치 2장
- 확신도 low 표시
- 틀리면 파일명 지정



확인 순서대로 눌러 보고 어긋난 점을 그대로 말한다

수정 요청도 프롬프트다 — 증상과 기대 결과를 한 문장씩

누구	사람이 확인 → Claude가 수정

어디	브라우저 → Code 탭 입력칸

확인 순서

- 차량 선택
- 견적만 선택 → 날짜 사라짐
- 시승+견적 → 두 영역 보임
- 필수 동의 빼고 제출 → 오류
- 390px 폭으로 줄여 보기

[입력] 프롬프트 · Claude Code (수정 요청 예)

```
모바일 390px에서 신청 버튼이 가려진다.
그 폭으로 재현하고 수정해줘.

신청 유형을 바꿔도 선택한 차량과
연락처 입력은 유지되게 해줘.

견적만 골랐는데 시승 날짜가 보인다.
quote일 때 시승 영역을 숨기고
검증에서도 빼줘.

Range Rover 상세에 Sport 사진이 나온다.
image_03.jpg는 range-rover-sport로 고쳐줘.
```

03

STEP 3–6 · 데이터

DB 저장·로그인·중복 방지

화면을 실제로 작동하는 MVP로 바꾼다

STEP 3

Supabase 프로젝트 생성과 키 입력은 사람이 한다

Supabase = PostgreSQL 데이터베이스 + 로그인을 제공하는 서비스

누구 사람이 직접 한다

어디 브라우저 supabase.com/dashboard

① 로그인

supabase.com

계정을 만들고 대시보드에 들어간다

② 새 프로젝트

jlr-experience-dev

이름에 dev를 붙여 테스트용임을 표시한다

③ DB 비밀번호

비밀번호 관리자

생성 화면의 비밀번호를 따로 보관한다

④ URL·키 확인

프로젝트 설정

Project URL, publishable key, secret key를 찾는다

키 값은 대화창에 붙이지 않는다 — 다음 장 프롬프트로 만든 .env.local에 직접 넣는다

STEP 3

환경 파일 틀은 Claude가 만들고 값은 사람이 넣는다

secret 키는 RLS를 우회할 수 있어 서버 전용 모듈에서만 쓴다

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
3단계: Supabase 연결 준비.
.env.example과 .env.local 빈 템플릿을 만들고
변수마다 Supabase 어느 화면 값인지 알려줘.

NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=
SUPABASE_SECRET_KEY=
NEXT_PUBLIC_APP_URL=http://localhost:3000
NOTIFICATION_MODE=log

secret 키는 server-only 모듈에서만 쓴다.
legacy 키(anon/service_role)면 차이를 설명해줘.
값은 출력하지 말고 내가 넣을 때까지 기다려.
```

[출력] Claude가 만드는 것

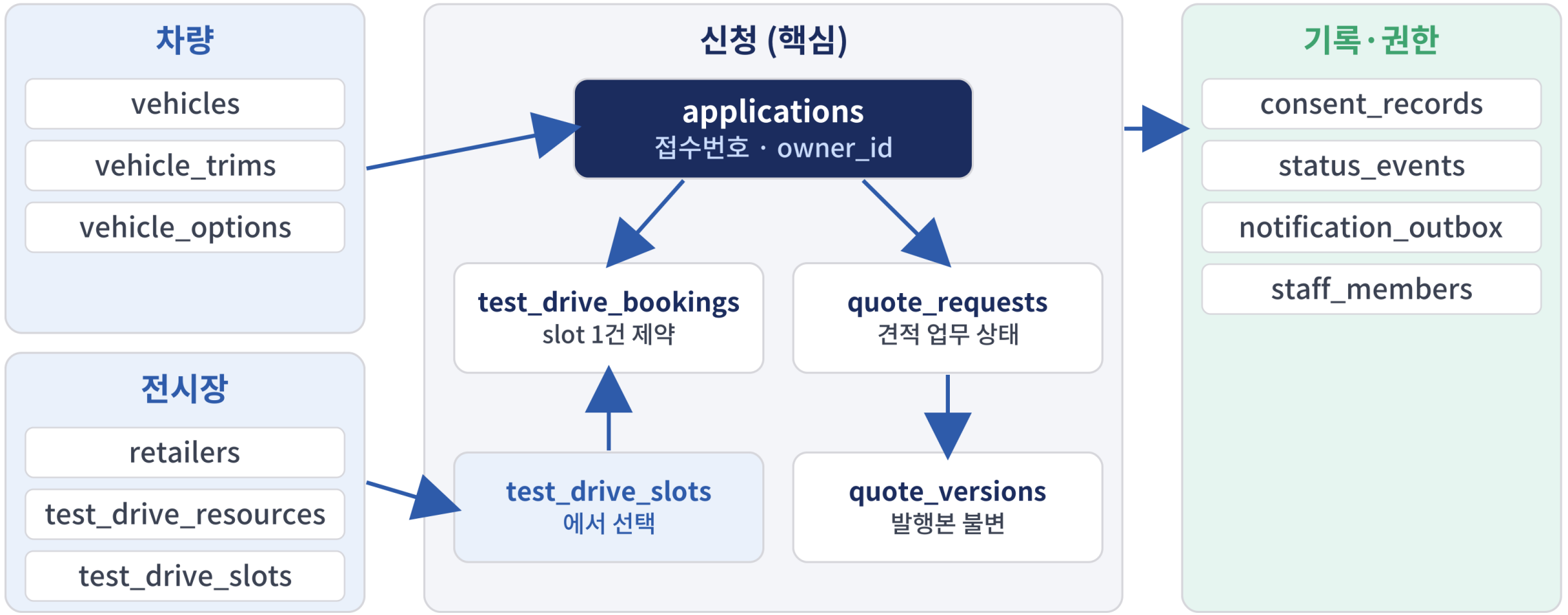
```
.env.example · .env.local(빈 값)
.gitignore에 .env.local
변수별 위치 안내
```

피드백 · 확인 → 수정 요청

- .env.local을 열어 값 직접 입력
- '값을 넣었다. 출력 없이 연결만 확인해줘'

테이블은 차량·전시장·신청·기록 네 묶음이다

applications 하나에 시승 예약과 견적 요청이 각각 매달린다



STEP 4

DB 구조와 권한은 migration 파일로 만든다

migration = DB 변경을 순서대로 기록한 SQL 파일, RLS = 행 단위 접근 권한

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
4단계: DB migration과 가상 seed를 만들어줘.
테이블은 차량·전시장·신청·기록 4묶음.
seed는 Kit의 data/*.csv로 만든다(idempotent).

- 모든 신청은 인증 사용자(owner_id)에 연결
- 상태 전이는 business_rules.md대로 서버 검사
- 개인정보 테이블 RLS, 비로그인 접근 금지
- 고객: 본인 신청·발행 견적만 / 직원: 담당 지점
- 내부 메모·감사 로그·미발행 견적은 비공개
- 역할 자기 지정 불가, 첫 관리자 지정 절차는
  docs/admin-setup.md
CLI 연결이 없으면 실행한 척하지 말고
SQL Editor용 파일과 docs/db-setup.md를 줘.
```

[출력] Claude가 만드는 것

```
supabase/migrations/*.sql
supabase/seed.sql
docs/db-setup.md · admin-setup.md
```

피드백 · 확인 → 수정 요청

- 다음 장 순서로 SQL 적용
- 오류는 전문 그대로 붙여넣기
- 같은 SQL 반복 실행 금지

SQL은 정해진 순서로 한 번씩 실행한다

Claude가 CLI로 직접 적용했다면 이 장은 건너뛴다

누구	사람이 직접 한다

어디	Supabase 대시보드 · SQL Editor

① 새 쿼리

SQL Editor

New query를 연다

② migration

db-setup.md 순서

파일 전체를 복사해
붙여넣고 Run

Success 확인

③ seed

seed.sql

가상 데이터를 넣는다

④ 확인

Table Editor

vehicles 6행, retailers
4행이 보인다

테이블이 보인다고 권한까지 끝난 것은 아니다 — 권한은 STEP 5에서 교차 접근으로 확인한다

STEP 5

이메일 인증한 사람만 자기 신청을 본다

OTP = 이메일로 받은 일회용 숫자 코드. 이름·전화번호·접수번호만으로는 조회하지 않는다

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
5단계: Supabase Auth 이메일 OTP 로그인.
- 신청 입력 → 숫자 코드 인증 → 제출 순서
- 인증 전 초안은 메모리에만(새로고침 안내)
- 소유자는 서버 세션의 user id·인증 이메일로
  정한다. body의 owner_id·이메일은 믿지 않는다
- /my는 본인 신청만, /my/[id]는 본인 상세만
- 접수번호는 표시용, 인증 수단이 아니다
- 관리자도 staff_members를 서버에서 확인
- Site URL·Redirect URL 설정은
  docs/auth-setup.md에 클릭 순서로
인증 우회 공개 API는 만들지 마.
다른 계정 접근이 403/404인지 테스트해줘.
```

[출력] Claude가 만드는 것

```
로그인·OTP 화면
/my · /my/[id]
docs/auth-setup.md
```

피드백 · 확인 → 수정 요청

- A로 접수 → 로그아웃
- B로 로그인 → A 상세 주소 직접 열기
- 내용이 안 보여야 통과

같은 시간 신청 두 건 중 한 건만 DB가 받아들인다

pending 접수도 자리를 차지한다 — 24시간 안에 확정되지 않으면 만료되어 자리가 풀린다



화면에서 막는 것은 보조 — 마지막 판정은 DB 제약이 한다

STEP 6

접수는 한 트랜잭션으로 저장하고 중복은 DB가 막는다

트랜잭션 = 여러 저장을 모두 성공하거나 모두 취소하는 묶음, idempotency key = 재시도 식별 번호

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
6단계: 가상 adapter를 DB로 바꾸고 접수 API.
서버 검증: 슬롯은 test_drive/both만, 견적 조건은
quote/both만. 휴무·지난 시간·비활성 슬롯·
잘못된 옵션 조합 거절. 가격은 서버가 계산.
중복 방지: 자원별 시간 겹침은 exclusion 제약,
활성 예약(pending/confirmed)은 슬롯당 1건.
pending은 24시간 뒤 expired로 명시 업데이트
(관리자 작업 + 스케줄러 연결점). 확정은 만료 없음.
신청·시승·견적·동의·outbox는 한 DB 함수로.
idempotency key: 더블클릭 재시도는 같은 결과.
슬롯 충돌은 409 + '시간을 다시 선택해 주세요'.
통합 테스트로 접수번호·상태가 DB와 같은지 확인.
```

[출력] Claude가 만드는 것

```
접수 API · DB 함수
제약 migration
통합 테스트 결과
```

피드백 · 확인 → 수정 요청

- 같은 슬롯 동시 신청 → 1건만
- 견적만 신청은 슬롯 미점유

04

STEP 7–9 · 업무 처리

관리자 처리·견적·알림

담당자가 시승을 확정하고 견적을 발행한다

STEP 7

관리자 화면에서 두 업무를 각각 처리한다

관리자 계정은 Supabase 대시보드 로그인과 다른, 앱 안의 직원 계정이다

누구 사람이 붙여넣고 Claude가 수행

어디 Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
7단계: DB에 연결된 관리자 화면을 완성해줘.
- /admin 요약: 오늘 접수·대기 시승·미처리 견적
- 목록: 유형·시승/견적 상태·기간·전시장 필터
- 상세: 고객·차량 구성·연락 방식, 담당자, 메모
- 시승 상태와 견적 상태는 별도 버튼으로 변경
- 슬롯 생성·비활성화·휴무 처리
- 동시 편집은 version으로 감지 → 새로고침 안내
- 보호 API는 서버가 역할·지점 범위를 검사
- 변경 주체·전후 상태·시각을 status_events에
공개 가입에서 role=admin 지정은 불가하게.
Kit의 staff.csv 계정으로 테스트 절차를 줘.
```

[출력] Claude가 만드는 것

```
/admin 화면 4종
docs/admin-setup.md
직원 계정 테스트 절차
```

피드백 · 확인 → 수정 요청

- 시승 확정 → 고객 /my에 반영
- 일반 계정으로 관리자 API 호출 → 거절
- 해운대 직원에게 강남 신청 안 보임

견적 합계는 서버의 가격표로만 계산한다

가격은 키트의 가상 가격표 — 브라우저가 보낸 총액은 믿지 않는다

$$\text{차량 견적 합계} = \underbrace{P_{\text{기본}}}_{\text{기본가}} + \underbrace{P_{\text{트림}}}_{\text{트림}} + \underbrace{P_{\text{색상}}}_{\text{색상1개}} + \sum_{k} \underbrace{P_{\text{옵션},k}}_{\text{선택 옵션}} - \underbrace{D}_{\text{승인 할인}}$$

손계산 검산 — docs/quote_sample.md의 Defender 110 견적

$$\underbrace{120{,}000{,}000}_{\text{기본가}} + \underbrace{12{,}000{,}000}_{\text{트림}} + \underbrace{1{,}500{,}000}_{\text{색상}} + \underbrace{4{,}000{,}000}_{\text{옵션2개}} - \underbrace{1{,}375{,}000}_{\text{할인1\%}} = \mathbf{136{,}125{,}000}$$

할인 1% 계산

- 137,500,000 × 1% = 1,375,000원

합계에 넣지 않는 것

- 취득세·등록비·탁송료 → 별도 확인

STEP 8

견적서는 담당자가 검토해 발행하고 버전으로 관리한다

견적 신청 = 고객의 상담 요청, 견적서 = 담당자가 검토해 발행한 결과

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
8단계: 견적 초안 → 담당자 검토 → 발행.
Kit의 docs/quote_policy.md를 그대로 따른다.
- 합계 = 기본가+트림+색상+옵션−승인 할인
  (원 정수, 중복 합산·음수 금지)
- 할인은 역할별 한도, 초과 시 상위 승인
- 취득세·등록비 등은 '별도 확인'(계산 금지)
- issued 버전은 불변, 수정은 새 버전
- 고객에게는 발행된 버전만 보인다
- 할부·리스는 '상담 희망' 저장,
  월 납입금 확정 표시 금지
- 인쇄용 견적 페이지 + print CSS
- 모든 화면·PDF에 '교육용 가상 견적'
quote_sample.md와 같은 금액이 나오는지 검산.
```

[출력] Claude가 만드는 것

```
견적 초안·발행 화면
견적 버전 v1·v2
인쇄 → PDF 저장
```

피드백 · 확인 → 수정 요청

- 발행 전 고객 /my에 안 보임
- 합계 136,125,000원 일치
- PDF 한글·금액·페이지 정상

할부 시뮬레이터는 가정을 모두 드러낸 참고용으로만 만든다

원리금균등 상환 — 금융사 승인 견적이 아니므로 확정 금액으로 표시하지 않는다

월 납입금 $M = \dfrac{P\,i\,(1+i)^n}{(1+i)^n - 1}$, $\quad i = \dfrac{r}{12}$, $\quad r = 0 \Rightarrow M = \dfrac{P}{n}$

P : 원금 $\quad$ r : 연이율 $\quad$ n : 개월 수 $\quad$ i : 월이율

결과 예시 (원금 6천만 원, 36개월)

- 연 6% → 월 약 1,825,316원
- 연 0% → 6천만 ÷ 36 = 1,666,667원
- 금리·기간·선수금 가정을 화면에 표시

[입력] 프롬프트 · Claude Code (선택)

```
할부 시뮬레이터를 참고용으로 추가해줘.
원리금균등 공식, r=0이면 P/n.
금리·기간·선수금 가정을 모두 표시하고
'금융사 승인 견적 아님'을 붙여줘.
리스 계산은 만들지 마.
```

STEP 9

알림은 먼저 기록 모드로 만들고 실제 발송은 나중에 붙인다

outbox = 보낼 알림을 먼저 DB에 적어 두는 대기열

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
9단계: 알림을 NOTIFICATION_MODE=log로 구현.
외부 발송 없이 notification_outbox에 기록하고
관리자 화면에서 내용을 볼 수 있게 한다.

사건: 접수(확정 전 안내), 시승 확정·취소, 견적 발행
- 사건별 unique key로 중복 방지, 재시도·최종 실패
- 알림 실패를 접수 실패로 표시하지 않는다
- 임의 수신자에게 보내는 공개 엔드포인트 금지
- 로그에 전체 전화번호·키·인증코드 금지

실제 메일 연결 절차는 docs/notifications.md에만.
```

[출력] Claude가 만드는 것

```
outbox worker
관리자 알림 로그 화면
docs/notifications.md
```

피드백 · 확인 → 수정 요청

- 시승 확정 → 로그 1건
- 같은 사건 두 번 → 로그 1건
- 문자·알림톡은 별도 계약 영역

05

STEP 10–11 · 검증·배포

테스트와 교육용 배포

자동 테스트 → 사람 확인 → 테스트 링크

STEP 10

권한·동시성·오류 시나리오를 자동 테스트로 검증한다

Playwright = 브라우저를 자동으로 눌러 보는 테스트 도구

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
10단계: docs/acceptance-checklist.md 시나리오를
Playwright UI 테스트와 DB 통합 테스트로 검증.
개발 DB의 테스트 레코드만 쓴다.
- test_drive 접수→확정→고객 상태 반영
- quote만: 시승 행 없음 / both: 상태 두 줄
- 같은 슬롯 동시 2건 → 1건만 성공
- 같은 idempotency key 재시도 → 중복 없음
- 사용자 B는 A의 신청·견적 접근 불가
- 일반 사용자의 관리자 API 거절
- DB 실패 시 완료 화면 없음·입력 보존
- 390px 모바일 잘림·가로 스크롤 없음
lint·타입·테스트·빌드 후 docs/test-report.md에
통과/실패/미실행을 나눠 기록해줘.
```

[출력] Claude가 만드는 것

```
tests/ e2e · 통합
docs/test-report.md
사람 확인용 클릭 순서
```

피드백 · 확인 → 수정 요청

- '미실행'이 통과로 적혔는가
- 실패 항목은 '하나씩 고쳐줘'

자동 테스트 뒤에 사람이 아홉 가지를 직접 확인한다

브라우저 두 개(일반 창·시크릿 창)로 고객 A·B와 관리자를 오간다

확인	기대 결과
견적만 신청	시승 날짜 없이 접수
접수 완료 화면	담당자 확인 전임을 표시
새로고침·다른 브라우저	인증 후 저장된 신청 확인
같은 시간 동시 신청	한 건 성공, 다른 건 재선택 안내
다른 사용자 상세 주소	정보 노출 없음
일반 사용자 관리자 접근	접근 거절
견적 발행 전/후	발행 후에만 고객 공개
PDF 저장	한글·금액·페이지 정상
알림 오류	접수 유지, 실패는 관리자가 확인

배포는 저장소·호스팅·인증 주소 세 곳을 잇는 일이다

각 칸 위는 누가 하는지 — 로그인과 값 입력은 사람, 점검과 검증은 Claude



서버 secret 변수에는 NEXT_PUBLIC_을 붙이지 않는다

STEP 11

배포는 준비 점검을 먼저 받고 실행한다

GitHub = 코드 저장소, Vercel = 저장소를 웹 주소로 배포하는 서비스

누구	사람이 붙여넣고 Claude가 수행	어디	Claude 데스크톱 앱 · Code 탭 입력칸

[입력] 프롬프트 · Claude Code

```
11단계: GitHub + Vercel 테스트 배포를 준비해줘.
- .env.local·비밀키·개인정보가 Git에 없는지
- production build 성공, lockfile 보관
- README: 실행·환경변수·migration·직원 등록
- Vercel 서버 secret에 NEXT_PUBLIC_ 없음
- Supabase Auth Site/Redirect URL을 배포 주소로
- NOTIFICATION_MODE=log, 교육용 표시 유지
- 알림 worker·pending 만료는 인증된 스케줄러

내가 할 GitHub·Vercel 클릭 순서를 알려주고
아직 배포하지 말고 준비 결과만 보여줘.
```

[출력] Claude가 만드는 것

```
배포 점검표
README.md
사람용 클릭 순서
```

피드백 · 확인 → 수정 요청

- 점검표에 빨간 항목이 없는가
- GitHub·Vercel 로그인은 직접

STEP 11

배포 후에도 시승과 견적을 한 건씩 다시 접수해 본다

배포 주소에서 다시 확인해야 로컬에서만 되는 문제를 잡는다

누구 사람 로그인 · Claude 실행·검증

어디 Code 탭 + GitHub·Vercel 화면

[입력] 프롬프트 · Claude Code

```
준비한 설정으로 교육용 테스트 배포를 진행해줘.
로그인·환경변수 입력은 내가 하도록 안내해줘.

배포 후 시승·견적을 가상 데이터로 각각 접수해
저장과 관리자 확인까지 검증하고,
로그인 권한·비밀키 노출·모바일 화면도 확인해줘.

실행하지 못한 검증은 따로 알려줘.
```

[출력] Claude가 만드는 것

```
https://...vercel.app
배포 검증 결과
미실행 검증 목록
```

피드백 · 확인 → 수정 요청

- 휴대폰으로 배포 주소 열기
- Supabase Redirect URL 반영 확인
- 환경변수 바꾸면 재배포

막히면 증상을 붙여넣고 우회 금지를 덧붙인다

그대로 복사해 쓰는 네 가지 프롬프트

[입력] 프롬프트 · Claude Code · 오류가 났다

```
아래 오류가 났다. (키·고객정보는 지움)
[오류 전문 붙여넣기]
재현하고 최소 변경으로 고친 뒤
검증을 다시 실행해줘.
DB 삭제·초기화로 우회하지 마.
```

[입력] 프롬프트 · Claude Code · 접수가 안 보인다

```
접수 완료는 떴는데 관리자 목록에 없다.
접수번호 기준으로 DB 저장·프로젝트·
환경변수·소유자·RLS·지점 범위·필터 점검.
서버 저장 없이 성공을 띄우는 코드도
찾아줘. 개인정보는 출력하지 마.
```

[입력] 프롬프트 · Claude Code · RLS 오류

```
'new row violates row-level security'
오류가 난다. 세션 전달·owner_id 설정 주체·
정책·함수 권한을 확인해줘.
RLS를 끄거나 전체 허용으로 바꾸지 마.
A/B 교차 접근 테스트도 실행해줘.
```

[입력] 프롬프트 · Claude Code · 다음 날 이어 하기

```
CLAUDE.md, docs/requirements.md,
docs/test-report.md, Git 변경을 읽고
완료/미완료 단계를 요약한 뒤
멈춘 단계부터 이어가줘.
새 프로젝트로 덮어쓰지 마.
```

진행 상태는 근거와 함께 보고받고 인수인계 문서로 남긴다

판정마다 실제 실행 근거를 요구한다

[입력] 프롬프트 · Claude Code · 어디까지 됐나

```
현재 결과를 아래 네 칸으로 나눠 보고해줘.

  화면만 됨
  DB 저장 됨
  권한 검증 됨
  배포 검증 됨

각 판정의 실제 실행 근거와 남은 일을 적고,
내가 브라우저에서 확인할 주소와
클릭 순서를 5개 이내로 안내해줘.
```

[입력] 프롬프트 · Claude Code · 인수인계

```
docs/handover.md를 작성해줘.
1. 고객/관리자 흐름
2. 로컬 실행 명령과 주소
3. 환경변수 이름과 위치(값 제외)
4. migration·seed·첫 직원 등록
5. 슬롯·휴무 관리
6. 가격표 갱신과 새 견적 버전
7. 알림·만료 작업과 장애 확인
8. 테스트 결과와 배포 검증 범위
9. 알려진 제한·미구현
10. 백업·권한 회수·개인정보 삭제
```

운영 전환에는 실제 담당자의 자료가 먼저 필요하다

교육용 MVP의 동의 필드 설계는 법적 적합성 확인을 대신하지 않는다

담당자가 줄 자료

- 사용 권한 있는 이미지·브랜드 자산
- 검증된 가격표와 기준일
- 전시장·시승 차량·운영시간
- 동의 문안·보유기간
- 견적 승인 정책·발신자 설정

[입력] 프롬프트 · Claude Code

```
교육용 시스템을 실제 운영으로 바꾸기 위한
차이만 정리해줘.
실제 담당자가 줄 자료와 코드 작업을 나눠줘.

작업: 테스트 데이터 분리, 백업/복구 확인,
개인정보 보유기간 삭제 작업, 관리자 MFA와
권한 회수, 알림 실패 처리, 스케줄러,
rate limit, 접수·견적 모니터링, 장애 연락 경로.

받지 못한 값을 상상해서 채우지 말고
항목별 '입력 대기'로 표시해줘.
```

사람은 묻고 확인하며 Claude Code는 만들고 검증한다

이번 구현에서 기억할 세 가지

1

한 단계에 한 프롬프트, 결과를 확인한 뒤 다음으로

누구·어디·입력·출력·피드백의 다섯 칸을 매번 반복한다

2

규칙은 문서로 주고 Claude가 읽게 한다

업무 규칙·견적 정책·동의서·가상 데이터를 키트로 넘기면 프롬프트가 짧아진다

3

저장·권한·중복 차단을 확인해야 완성이다

교차 접근, 동시 신청, 발행 전 비공개를 사람이 직접 눌러 본다

다음 확장: 실제 이메일 발송 · 문자/알림톡 · 보상판매 시세 조회 · 재고 연동